
DCL – Teil 1

BITLC | Tom Selig
2. September 2026

Aufgaben der DCL

- Die DCL ist eine der fünf Untergruppen von SQL.
- Mit ihr werden Server-Logins und Datenbank-User verwaltet.
- Den Usern können dann:
 - Server-Logins zugewiesen werden
 - Server- und Datenbank-Rollen zugewiesen werden
 - Rechte für Datenbank-Objekte erteilt und entzogen werden.
- Die Zuteilung bzw. Verweigerung von Rechten innerhalb einer Datenbank ist wichtiger Bestandteil eines Sicherheitskonzepts.
 - Jeder Benutzer darf nur das können, was er auch wirklich braucht.

SQL Server-Rechtesystem

- Der SQL Server unterscheidet grundsätzlich zwischen:
 - Server-Anmeldung (Login)
 - Mit einer Server-Anmeldung wird lediglich der Zugriff auf den Datenbank-Server ermöglicht.
 - Datenbank-Benutzer (User)
 - Ein Datenbank-Benutzer gilt nur innerhalb der Datenbank in der er erstellt wurde. Er ermöglicht den Zugriff auf die Objekte der Datenbank.
- Es wird also immer sowohl ein Login als auch ein User benötigt.
- Sowohl für den Login als auch für den User können verschiedene Rechte verwaltet werden.

Server-Anmeldung

- Bei der Server-Anmeldung wird unterschieden zwischen:

Windows-Authentifizierung

Bei dieser Variante werden Benutzer und Passwörter vom lokalen Betriebssystem verwaltet.

Eine separate Anmeldung am SQL Server entfällt, weil das Betriebssystem den angemeldeten User an den SQL Server weitergibt.

SQL Server-Authentifizierung

Bei dieser Variante werden Benutzer und Passwörter vom SQL Server verwaltet.

Die Informationen dazu werden in verschlüsselter Form in System-Tabellen abgelegt.

Login verwalten

- Ein neues SQL Server- oder Windows-Login kann mit dem Befehl **CREATE LOGIN** erstellt werden:

Syntax: **CREATE LOGIN** <name> **WITH PASSWORD** = <password>;
 CREATE LOGIN <domäne\konto> **FROM** WINDOWS;

Beispiel: **CREATE LOGIN** ProjektDBRW **WITH PASSWORD** = 'PDBRW';

- Da das Login nur für den Server gilt, kann es aus jeder Datenbank heraus erstellt werden. Es erscheint anschließend im Objekt-Explorer unter dem Pfad '*Sicherheit/Anmeldungen*'.
- Mit **ALTER LOGIN** kann ein Login nachträglich verändert werden.
- Mit **DROP LOGIN** kann ein bestehendes Login gelöscht werden.

User verwalten

- Bei der Anlage und Verwaltung von Usern ist die aktuell ausgewählte Datenbank von Bedeutung, da User immer nur innerhalb einer Datenbank gültig sind.
- Mit dem **CREATE USER**-Befehl können User für eine Datenbank erzeugt werden. Dabei gibt es zwei Möglichkeiten:

Der User wird mit dem Namen eines Logins erstellt. Dann sind User und Login automatisch miteinander verbunden.

Der User wird mit einem abweichenden Namen erstellt und einem bestimmten Login zugeordnet. Dann hat das Login in der Datenbank einen Alias-Namen.

User verwalten

- Erstellen eines Users mit dem gleichen Namen wie ein Login:
Syntax: `CREATE USER <login>;`
Beispiel: `CREATE USER ProjektDBRW;`
- Erstellen eines Users mit abweichendem Namen für ein Login:
Syntax: `CREATE USER <name> FOR LOGIN <login>;`
Beispiel: `CREATE USER ProDBRW FOR LOGIN ProjektDBRW;`
- Die zweite Variante funktioniert immer, auch wenn `<name>` und `<login>` identisch sind.
- Jedes Login kann nur einen User in einer Datenbank haben.

Rechte für User und Rollen

- Der neue angelegte User kann jetzt auf die Datenbank zugreifen.
- Allerdings kann er in der Datenbank immer noch keine Aktionen durchführen, da ihm noch keine Rechte erteilt wurden.
- Rechte können im SQL Server sowohl für einzelne User als auch für Rollen vergeben werden.
- Rollen werden genutzt, um die Rechtevergabe an die User zu vereinfachen und sicherer zu gestalten.
- Rollen sind eine Zusammenfassung von mehreren Rechten. User können einer oder auch mehreren Rollen zugeordnet werden und erben dann die entsprechenden Rechte dieser Rollen.

Rechte für User und Rollen

- Im SQL Server können Rechte auf drei Arten vergeben werden:

Berechtigung gewähren (GRANT):

Mit **GRANT** gewährt man einem User das Recht, eine bestimmte Aktion auf einem Datenbank-Objekt auszuführen.

Berechtigung verweigern (DENY):

Mit **DENY** verweigert man dem User bestimmte Aktionen durchzuführen. Ein **DENY** ist immer stärker als ein **GRANT**.

Berechtigung entziehen (REVOKE):

Mit **REVOKE** entzieht man ein zuvor erteiltes **GRANT** oder **DENY** wieder. Das Recht wird auf einen neutralen Zustand gesetzt.

Rechte für User und Rollen

- Beispiel: Rechtevergabe für eine Tabelle:

	SELECT	INSERT	UPDATE	DELETE
Benutzer	GRANT	REVOKE	REVOKE	REVOKE
Rolle 1	REVOKE	REVOKE	GRANT	GRANT
Rolle 2	REVOKE	REVOKE	REVOKE	DENY
Ergebnis	ja	nein	ja	nein

- Der Benutzer selbst hat nur das Recht für **SELECT**. Über die Rolle 1 erbt er zusätzlich das Recht für **UPDATE** und **DELETE**. Das Recht für **DELETE** wird ihm aber durch Rolle 2 wieder verweigert, da ein **DENY** immer ein **GRANT** schlägt. Das Recht für **INSERT** hat er nicht, da es ihm nirgendwo explizit zugesprochen wird.

Rechte für User

- Rechte können Usern für ein Objekt direkt zugewiesen werden:

Syntax: <vergabeart> <berechtigung> [, ...]
 ON <objekt>
 TO <empfänger> [, ...];

Beispiel 1: GRANT SELECT, INSERT, UPDATE
 ON Mitarbeiter
 TO ProjektDBRW;

Beispiel 2: DENY DELETE
 ON SCHEMA::dbo
 TO ProjektDBRW, AndererUser;

- Für die UPDATE- und DELETE-Rechte ist immer auch das SELECT-Recht erforderlich, da die Datensätze lesend ausgewählt werden.

Rechte für Rollen

- Neue Datenbank-Rollen können mit dem **CREATE ROLE**-Befehl erstellt werden:

Syntax: **CREATE ROLE** *<name>*;

Beispiel: **CREATE ROLE** MitarbeiterCRUD;

- Mit dem **ALTER ROLE**-Befehl können User zu einer Rolle hinzugefügt oder wieder entfernt werden:

Syntax: **ALTER ROLE** *<name>*
ADD|DROP MEMBER *<user>*;

Beispiel: **ALTER ROLE** MitarbeiterCRUD
ADD MEMBER ProjektDBRW;

Rechte für Rollen

- Die Vergabe von Rechten für Rollen funktioniert analog zur Vergabe bei einzelnen Usern:

Beispiel 1: `GRANT SELECT, INSERT, UPDATE
ON Mitarbeiter
TO MitarbeiterCRUD;`

Beispiel 2: `DENY DELETE
ON Mitarbeiter
TO MitarbeiterCRUD;`

- Durch die Vergabe von Rechten für User und Rollen, können auch komplexe Szenarien relativ einfach abgebildet werden.
- Regel: So viele Rechte wie nötig, so wenig Rechte wie möglich.